COSC 6385 - Homework 1: Branch Prediction

Nicholas Anderson

1 Problem Statement

For this homework we have been tasked with creating a branch prediction class in c++. Template code was given, of which the important methods are its ability to read instructions and identify branch statements. We are given a multitude of different traces, and by using the *run* command we can find the average mispredictions-per-1000-instructions (MPKI) across all given traces.

We were tasked with implementing *branch outcome prediction* using a two-way global predictor, and a bimodal predictor. We were not required to implement loop prediction or branch target prediction. Essentially, we use a bimodal table, and a two-way global predictor to predict whether a branch will be true or not. Additionally, implementation of the path information register (PIR) and the hash access function was not required for the global predictor. To do this we make moifications to the `pm_predictor` class located in **my_predictor.h** header file.

2 Preliminary Results

Initially, the prediction is set to simply always predict that a branch is not taken. This results in an average MPKI of 86.311 across all given test data. The sample code also comes with the `gshare_predictor` class which acheives an average MPKI of 6.305, the output can be seen in figure 1.

```
traces/164.gzip/gzip.trace.bz2                  12.473
traces/175.vpr/vpr.trace.bz2                    13.415
traces/176.gcc/gcc.trace.bz2                    11.254
traces/181.mcf/mcf.trace.bz2                    15.837
traces/186.crafty/crafty.trace.bz2               5.837
traces/197.parser/parser.trace.bz2              10.008
traces/201.compress/compress.trace.bz2           7.831
traces/202.jess/jess.trace.bz2                   1.562
traces/205.raytrace/raytrace.trace.bz2           2.756
traces/209.db/db.trace.bz2                       3.909
traces/213.javac/javac.trace.bz2                 2.267
traces/222.mpegaudio/mpegaudio.trace.bz2         2.188
traces/227.mtrt/mtrt.trace.bz2                   2.657
traces/228.jack/jack.trace.bz2                   3.033
traces/252.eon/eon.trace.bz2                     1.807
traces/253.perlbmk/perlbmk.trace.bz2             2.554
traces/254.gap/gap.trace.bz2                     3.926
traces/255.vortex/vortex.trace.bz2               1.222
traces/256.bzip2/bzip2.trace.bz2                 0.094
traces/300.twolf/twolf.trace.bz2                21.489
average MPKI: 6.305
```

Figure 1: Trace MPKI results using the sample gshare output, which uses a table size of 2^{15}.

3 My Solution

My code can be found at [3], which is the github repository containing my homework solutions.

3.1 Bimodal Predictor

For my implementation I created a bimodal table which has 4096 entries, with each entry having a 2-bit counter. For my implementation I heavily referenced an online article [4]. Implementation details can be found in section 3.1.2.

3.1.1 Two-bit Counter

Since a 2-bit counter comes up in global prediction as well it is worth giving an explanation of its functionality. A 2-bit counter is exactly what it sounds like, a counter which is 2 bits. The purpose of this counter is to de-

termine what the branch prediction will be. The 4 possible states of a 2-bit counter are:

00: Strongly not taken
01: Weakly not not taken
10: Weakly taken
11: Strongly taken

Not taken states correspond to the a 'false' prediction, while taken states correspond to a 'true' prediction. Note that terminology is easy to confuse here: a 'false' prediction does not mean the branch predictor was incorect, but that the predictor is predicting that the condition will be false. If the condition is false in this case, then a predictor predicting 'false' is a correct prediction. The opposite is the case for 'true' predictions. I will try my best in the report to denote predictions as either true or false, and whether they were correct as correct or incorrect

Having a 2-bit counter allows branch prediction to change its stage on incorrect predictions, but not always between a true and false prediction. For bimodal prediction, after a prediction the counter state is updated, depending on the result of the prediction. For example we look at a true prediction (i.e. branch predictor predicts true). If this prediction is correct then the 1 is added to the counter, if it is incorrect then 1 is subtracted from the counter. For a false prediction, 1 is subtracted on a correct prediction and added on an incorrect one. If the counter state is already at 11 or 00 and 1 is added or subtracted respectively, then the counter is not updated. This implementation allows for predictions to be incorrect, and the state to be updated in a way where it will make the same prediction, which can prove useful for many patterns.

Take a common nested for loop pattern where a for loop of size 2 is repeated indefinitely. The resulting branch pattern would look like:(T T F T T F T T F...). Assuming a one bit counter that starts with true, the first 2 trues would be predicted correctly, then the false would be a misprediction. On the misprediction the counter would change to false, then mispredict the next true as well, finally it would predict the 5th prediction correctly as true, then the 6th and 7th prediction incorrectly. This pattern would maintain for the rest of the pattern, resulting in an asymptotical prediction accuraccy of 33% (predicts the first true incorrectly, the second true correctly, and the false incorrectly). With a 2-bit counter we can increase the accuracy to 66%. We start at **10**, weakly taken. The first prediction is true so the counter is incremented to strongly true **11**, strongly taken. The second prediction is also true, but the counter is already **11** so we remain at strongly taken. On the first false prediction we have a misprediction, so we decrement our counter to **10**. However, we are still at weakly taken, so unlike the 1-bit counter we predict correctly (true) on the 4th prediction. In this scenario we continue to correctly predict all true values and incorrectly predict all false values, giving us asymptotical accuracy of 66%.

3.1.2 Bimodal Table Implementation

As stated earlier, loop branch prediction is not required, so the bimodal prediction outcome is simply the value from the leftmost bit from the 2-bit counter. Each table entry contains its own 2-bit counter. To find which table entry we examine for prediction, we take trailing bits from the instruction pointer (IP) address equal to log_2(Bimodal Table Size). In our implemenation we have a bimodal table size of 4096, so we take the 12 trailing (rightmost) bits from the instruction pointer. The table is implemented as an std::array of type `int8_t` (for simplicity the above examples relate to `uint8_t`, signed integers can be implemented just as easily). I use `int8_t` as that is the simplest method to implement a 2-bit variable while wasting minimal space, and it allows an initializtion to weakly true by simply invoking:

`array<int8_t, size> bimodal_table{};`
In my implementation values are always checked before they are incremented to ensure the 2bc value always remains between -2 and 1. All values are initialized as 0, which corresponds to a weakly taken state. Using only the bimodal table for prediction, my results indicate an average MPKI of 10.267, which can be seen in figure 2.

```
traces/164.gzip/gzip.trace.bz2                  15.031
traces/175.vpr/vpr.trace.bz2                    14.673
traces/176.gcc/gcc.trace.bz2                    19.313
traces/181.mcf/mcf.trace.bz2                    33.271
traces/186.crafty/crafty.trace.bz2              13.097
traces/197.parser/parser.trace.bz2              15.420
traces/201.compress/compress.trace.bz2           9.368
traces/202.jess/jess.trace.bz2                   8.773
traces/205.raytrace/raytrace.trace.bz2           3.704
traces/209.db/db.trace.bz2                       6.437
traces/213.javac/javac.trace.bz2                 3.198
traces/222.mpegaudio/mpegaudio.trace.bz2         3.166
traces/227.mtrt/mtrt.trace.bz2                   3.682
traces/228.jack/jack.trace.bz2                   7.175
traces/252.eon/eon.trace.bz2                     9.443
traces/253.perlbmk/perlbmk.trace.bz2             5.848
traces/254.gap/gap.trace.bz2                     9.760
traces/255.vortex/vortex.trace.bz2               1.930
traces/256.bzip2/bzip2.trace.bz2                 0.113
traces/300.twolf/twolf.trace.bz2                21.943
average MPKI: 10.267
```

Figure 2: Trace MPKI results using only bimodal prediction with a 2-bit counter and a table size of 2^{12}.

3.2 Global Predictor

For this assignment we implement a 2-way global predictor, with each 512 elements each way. As described in the assignment, we are not required to implement the PIR or a hashing function. We use a global history vector of 15 bits to keep track of recent branch paths, allowing us to retain information on the past 15 paths. To find the global predictor table entry, we take bits [14:6] of the instruction pointer (inclusive), and perform a bitwise XOR operation with bits [14:6] of the global history vector. This value is the index we examine inside the global table. A tag is also created and stored inside the global table, its value being the result of a bitwise XOR between [5:0] of the instruction pointer and the global history. This value is used to determine whether there is a global predictor hit. If the tag inside the global predictor table at the index matches the tag from the branch condition, for any way, then we determine that there is a global predictor hit and we use the 2-bit counter from this table to determine our prediction. If the tag does not match the value in any way, then we fall back to the bimodal table. Each way contains its own tag. My implementation was heavily based off of a medium article [5] and a two chatGPT interactionsI had with GPT 5.6 [2] [1].

3.2.1 Global Predictor Implementation

For implementing the global table we initialize a 2-dimensional array (i.e. matrix) like so:
`uint8_t two_way_global_table[512][2];`
Each value is a 8 bit unsigned int, with the leading 6 bits used to contain the tag, and the 2 trailing bits containing the 2-bit counter. To perform operations on these we use bitmasks of `0x3F` and `0x3` respectively. Additionally we utilize a global valid table:
`uint8_t global_valid[512][2]={};`
with all values initialized at 0. This stops the global predictor table from initially using an unallocated prediction if the tag happens to match the uninitialized (garbage) value in `two_way_global_table`.

3.2.2 Global Predictor & Bimodal Table Results

With both branch prediction tables implemented, we see a MPKI of 6.557. The full output can be seen in figure 3. When compared to the `gshare_predictor` class, we see

that we have a worse MPKI of .25. While our results our slightly worse, it is important to note that our total table size is 512∗2+4096 = 5120 while the `gshare_predictor` has a table size of 32768, which is 6.4× larger. In contrast, our MPKI 1.04× larger. This shows that using both tables allows you to save table space while having similar MPKI compared to just using a global predictor table (which is what `gshare_predictor` implements).

```
traces/164.gzip/gzip.trace.bz2                14.306
traces/175.vpr/vpr.trace.bz2                  14.244
traces/176.gcc/gcc.trace.bz2                  10.531
traces/181.mcf/mcf.trace.bz2                  17.513
traces/186.crafty/crafty.trace.bz2             5.816
traces/197.parser/parser.trace.bz2             9.373
traces/201.compress/compress.trace.bz2         9.113
traces/202.jess/jess.trace.bz2                 1.207
traces/205.raytrace/raytrace.trace.bz2         2.316
traces/209.db/db.trace.bz2                     3.962
traces/213.javac/javac.trace.bz2               2.659
traces/222.mpegaudio/mpegaudio.trace.bz2       2.357
traces/227.mtrt/mtrt.trace.bz2                 2.406
traces/228.jack/jack.trace.bz2                 2.732
traces/252.eon/eon.trace.bz2                   1.761
traces/253.perlbmk/perlbmk.trace.bz2           2.253
traces/254.gap/gap.trace.bz2                   4.648
traces/255.vortex/vortex.trace.bz2             0.928
traces/256.bzip2/bzip2.trace.bz2               0.071
traces/300.twolf/twolf.trace.bz2              22.958
average MPKI: 6.557
```

Figure 3: Final results with both the bimodal and global precition tables.

3.3 Updating the Tables

Figure 4 describes our tabe update policy. We do not update both tables every instruction. Firstly, tables are only updated on a branch instruction. If the instruction is not a branch condition, then we automatically set branch prediction to true and perform no update. Secondly, we do not update the bimodal table when there is a global predictor hit, as the global table is used. On a global table hit, whichever way had the hit, we increment or decrement the counter in the same way as the bimodal table (described in section 3.1.1). On a global table miss, we update the

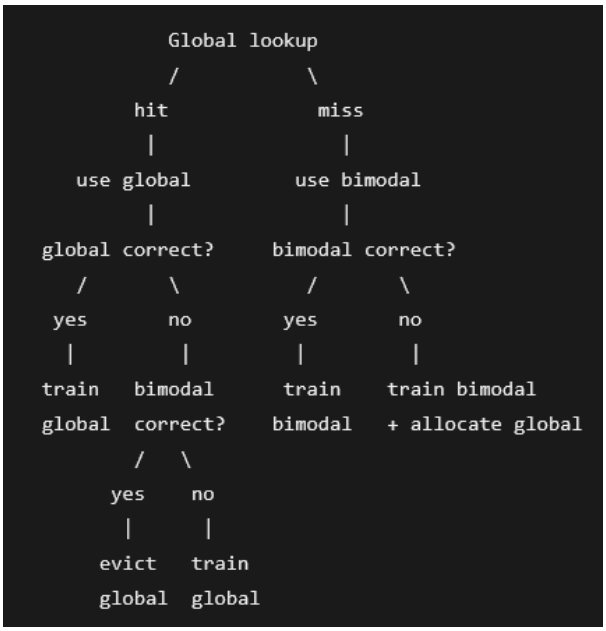


Figure 4: chatGPT generated diagram describing the table updating policy. 'train' corresponds to updating the 2-bit counter, and 'evict' corresponds to replacing the tag in the global table.

2-bit counter in the global table. On a global table miss we update the 2-bit counter in the bimodal table based on whether the branch was taken or not. If the bimodal table prediction was incorrect (on a global table miss), then we allocate the tag and initialize the 2-bit counter in the global table. When a tag is replaced in the global table, we reinitialize the counter to be either weakly taken or weakly not taken, based on whether the path was taken or not.

References

[1] chatGPT. chatgpt conversation about implementing a global branch predictor. https://chatgpt.com/g/g-p-6ab56d61a49c8191b907e193b870ee6d-cosc-6385-ca/c/6ac6a5e8-1c90-83e9-80e9-1b87a2f6ae5d, 2026. Accessed: 10-8-2026.

[2] chatGPT. chatgpt global prediction hit decision conversation. https://chatgpt.com/g/g-p-6ab56d61a49c8191b907e193b870ee6d-cosc-6385-ca/c/6ac69114-8ae8-83ea-aa39-24e3d0adf02d, 2026. Accessed: 10-8-2026.

[3] Nick123293. cosc-6385-computer-architecture. https://github.com/Nick123293/cosc-6385-Computer-Architecture, 2026. Accessed: 10-8-2026.

[4] The Arch Bytes: From Core to Code. Bimodal branch predictor. https://medium.com/@himanshu0525125/bimodal-branch-predictor-b20b8c31850c, 2024. Accessed: 10-8-2026.

[5] The Arch Bytes: From Core to Code. Gshare branch predictor. https://medium.com/@himanshu0525125/global-history-branch-predictor-0eddb6bdd001, 2024. Accessed: 10-8-2026.